SubTask 3.1: Completing the Group Session Initialization Protocol

To: kiranjain@gmail.com, misaonakano@gmail.com

Subject: Completing the Group Session Initialization Protocol

Dear Kirao and Misao,

I am writing this email to explain to you the steps in completing the group session initialization protocol.

First, there are a few missing steps at the end of Fig. 7. After performing the steps in Fig. 7, the user will have to perform a few more steps to ensure the group session maintains its integrity, authenticity and confidentiality.

Step 1: Extract the SESSID

As session IDs are unique, by extracting the session ID, the group members can identify their session and are able to distinguish it from other sessions. This ensures that the group members know that they are in the correct session.

Step 2: Verify the Signature of the Group Initiator

By ensuring that the signature of the group initiator is valid, other members will not be fooled by malicious group insiders pretending to be sending messages as other honest members. This maintains the group session's integrity and authenticity.

Step 3: Decrypt the gsk

As the gsk is used to encrypt and decrypt group messages during the session, by decrypting the gsk, group members can obtain the shared encryption key needed to communicate within the group securely. This allows the group members to communicate with one another effectively while also maintaining the confidentiality of the group session.

Next, I will explain how to extract the SESSID and decrypt the gsk of a message that "DeltaZhang00" sent to "ALICEKerr000".

Reading the Message

First, by using the command below, the contents of the message can be read.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ jq . 3_1_1-DeltaZhang00-to-ALICEKerr000-msg.out
```

Output:


```
{
  "SESSInit": {
    "cert_s": {
      "id": "DeltaZhang00",
      "pk": "-----BEGIN PGP PUBLIC KEY BLOCK-----\n\nmQENBGYQ6doBCAC5Vw7eyPPupsLmDStYlwNtEcenaEN2E3F4CWZ09gn7la5pp/0L\nI2RJkINJVFL5X
aD5kft+0JjMF5Imv9wIP6dxBDQp2kgRRgeztY4ahb5N/1yp0I4P\nbZOCO4/15IxgGQ0ObICKDkGIfTtf6HLHEFgK+AX3eIKhj8qyNfCymi07NXMsE8GU\ndJpsSj1o7s4SZ
37WYrz3xzxlWqHbRYLtOJl0la+o2EUDPvNf8ZHdBY59kRHjoQ2h\ns8nblLSG5AG0KSWRfBlxETunwLvRr9lvSunMQbfTyKLjAgPMnM88bKDoG0wbrB3g\nSPVvVN/yyY/v0
Qsom2W6w1IUZL8WQxf6whvtABEBAAG0K0RlbHRhWmhhbmcwMF9z\nIChEZWx0YVpoYW5nMDAgU2lnbmF0dXJlIEtleSmJAVQEEwEKAD4WIQRcal0cOi/L\nkFPg0wZvvzumH
AHnaQUCZhDp2gIbLwUJAeEzgAULCQgHAgYVCgkICwIEFgIDAQIe\nAQIXgAAKCRBvvzumHAHnaZmOB/9Y6wN8HQbtMQuNgq3SCOSs9OfOSD9PYvgDifwk\nNnQiMBmNYkgTl
enNz/X6oeggqXGy6MyOEYDVRZ+Zf75vj/+t0UYlRH6t/MPylIkV\nCPzOgglowGBtHD2iCyAoHu6MFudTYD31hYXYhcNJMxXKTs7yks/pqKARrC4zhMJA\ns9vKDzfips3xU
hGG4kqu64nkYzJnnmdkDUNeuFnhTSDPLDP1zkwN7QVNv0YPJmm0\nJO4pf6i4EIIi0/jrnzGhfYSiQqxGqIlol5zq4t3fVZdpbz50+awWLS1Vid4Ak7Ux\nenfyybPR5I124
Mhe3qMR+lS5WoWGkO84kaKdJ5JWAUuKD9ffmQENBGYRNNwBCADa\np+kYuhy4dyQKQZ6R5b1Qr5XxRSkJxViPGXvyyzDF8wMNjM8/R4C7dy1xxsVJLCIN\nxMUxtAyA4RMIS
DEeaFQurBnbcNAAl6VLx075fZELHmQAbvFGv0EqoZvNyUAmxbd0\nN7ujwsV2VrfMGdGw6spEfgVBoNE/m5MOn0Gs4RL5wlRFFxOZSLVBuixrkQrs16jx\n5hb0SP46lbwOQ
fruTxbwafKB0ypW30S6BvyNzJZd/Rn66Jjy70lRg+Wxv3jYVgbD\ntYFPsBPXaITfBYaGwplgRoOfqXdeaetCbkEONnRul3cofIJ0K6xLHhDYQCx3J8YU\n89yMjYNfg0EIN
Z3TxR3XABEBAAG0K0RlbHRhWmhhbmcwMF9zICh EZWx0YVpoYW5n\nMDAgU2lnbmF0dXJlIEtleSmJAVQEEwEKAD4WIQR1XzDlhSe5rQaAVe/tjPpCu/oZ\nKgUCZhE03AIbL
wUJAeEzgAULCQgHAgYVCgkICwIEFgIDAQIeAQIXgAAKCRDtjPpC\nu/oZKj8JB/9atz8iQjJbi7CnbnS6gcLCJ15jk12VgR+1BbCxPTmMMhaf4IhQEH+n\nNYeR81ydXT5ev
JnCVG4MOuwxTr1ht87B0SNSksArknFK+sX2ZNR3iYHcioxC7jYz\nfqLTk+2c21RCOLyU02ePeZTLsp7h3mdxZmRLWfHt3ztoK9IfM2Phvc1ds8uTpBeT\ni/qVG1qcQDUuK
/kmnyUbDn/8a2dyZ13ehV8PvDFRAnARUqgpt1unoIEadEV7jZgG\nG5X24hMVRn995axeNP723iKYU5NvxLfovoRdZTjiuR1mGF6ES7f5cnbl0ozICt+W\nECuyBS+a9vfCa
l8YuTjjftF5X6iviqwu\n=Y2ew\n-----END PGP PUBLIC KEY BLOCK-----",
      "sig": "-----BEGIN PGP SIGNATURE-----\n\niQEzBAABCgAdFiEEfXA6gwZkcUt9ejvZPBiG38xwm9MFAmYRNOEACgkQPBiG38xw\nm9MO2AgAjzS/1BHuD91
bCTSsYcNi7kJIfAaAoow8znXm+PaglBs8+Zrt+zVUBJ0N\nBtBzMkazldYVHcgSJNNoVzthE8tVLrC3Ev+erFVqz77d2NwqGO4wIm9WFi82VUy+\ntkqvUZ3lhUIOuZ6tObW
d9ecoYsEVSfMJHlPrjZ5fn0rSQ3VzsTZNiYrmudIgShRF\nAALAIaRCg3S2hxwdAHuWL14eKHRC7lQaoov7QGIkVat+W9lxS9QGym3mPwTs+H90\nnmw7Ooq8oK6m5OLRfrh
2XScGKv96XpOYc0VtR/ity2OXT0Kv/AGAKnhKegBDuHqA\nA2ReFFUENMwb3b5DfZuPgJcbcwhe5w==\n=JSRI\n-----END PGP SIGNATURE-----"
    },
    "sessid": "2465",
    "c_i": "-----BEGIN PGP MESSAGE-----\n\nhQEMA9aIh3UhmnlwAQf+MJYyBZCECKJE7rjVo/XD17wFYMj7CRD16t2dvJxV6t9N\nU/SsIIoWU5eJsnUdWiHCr76
R914Vk4HZY3smOT99gQIIfE0UOc6W2jCog1dxA/Y1\n2aiNVkCYwnlBNgjie0fPLeEiMrr9zM6AKycuXYve4R41nkfNFt3KEoRZvdDFuJwy\n2X5v4RNlX29y/PTa8bw/XNh
nfzhnnPRu/ZuqeamtqIRz4r1l+1lEbTQhmoNB2pv1\nKGI12EO6wjJXgq0JMbhMYwCv/C+62ndwSU1FrlNRnL295ojvd7XH0DtlKTtpdc9M\n+QNusY5q0XmDOK0AADdy7zw
CJlPhoLBUD4MeptU1+tJeATELmfiTAEU1ctZ9UYMz\nyOXZdMJ3RkllXquK0iYe76xB3CLcnzHfSqT31BpBm/nDQd+II0J3qSscJ+DxNeeZ\nP34QoaUkDXdAc6XjcIvQsw+
LkUsIoV7ZljU19qK4Vg==\n=GtJc\n-----END PGP MESSAGE-----\n",

    "sig_t": "-----BEGIN PGP SIGNATURE-----\n\niQEzBAABCgAdFiEEXGpdHDovy5BT4NMGb787phwB52kFAmYTPlgACgkQb787phwB\n52lMCAf6ArssvsvHCx3
2fMhjj3DTFRVr/D/a6c36T+68BRtYievXeu2WxolvYb0L\n7QJCRNO1HFXCuigdxsCKYDuRfjekjSXd8yOhEWEpmffGjvL59ZVnolIq69q/9b1h\nKjBJNmJ6PD04POORMhY
mGPJdS2zerOD3T5pzYz1MKkeRsa11PX2caePGoQwiLr+o\nqpp+oKt77cCeoXX3nnhxavW4aZbSo0wjaaWNdN5EBaWJfsr8Ac6TT4gxjOcfp+D7\ncFGlTSplVVEa88HqtNS
5QGwYgnUUM/58zU7m9DWSIwqHbL2LJ3o9lYh1jy8tqwse\nOISRNr1ENXWRGm33LOezAVQVVnKq3A==\n=x7tf\n-----END PGP SIGNATURE-----\n"
  }
}
```


Extracting the SESSID

The session ID is obtained by using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ jq .SESSInit.sessid 3_1_1-DeltaZhang00-to-ALICEKerr000-msg.out
"2465"
```

Session ID: 2465

Decrypting the gsk

The ciphertext sent by “DeltaZhang00” is obtained using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ jq .SESSInit.c_i  3_1_1-DeltaZhang00-to-ALICEKerr000-msg.out > Ci
phertext.txt
```

Output:


```
"-----BEGIN PGP MESSAGE-----\n\nhQEMA9aIh3UhmnlwAQf+MJYyBZCECKJE7rjVo/XD17wFYMj7CRD16t2dvJxV6t9N\nU/SsIIoWU5eJsnUdWlHCr76R914Vk4HZY3
smOT99gQIIfE0UOc6W2jCog1dxA/Y1\n2aiNVkCYwnlBNgjie0fPLeEiMrr9zM6AKycuXYve4R41nkfNFt3KEoRZvdDFuJwy\n2X5v4RNlX29y/PTa8bw/XNhnfzhnnPRu/Z
uqeamtqIRz4r1l+1lEbTQhmoNB2pv1\nKGIl2EO6wjJXgq0JMbhMYwCv/C+62ndwSU1FrlNRnL295ojvd7XH0DtlKTtpdc9M\n+QNusY5q0XmDOK0AADdy7zwCJlPhoLBUD4
MeptU1+tJeATELmfiTAEU1ctZ9UYMz\nyOXZdMJ3RkllXquK0iYe76xB3CLcnzHfSqT31BpBm/nDQd+II0J3qSscJ+DxNeeZ\nP34QoaUkDXdAc6XjcIvQsw+LkUsIoV7Zlj
U19qK4Vg==\n=GtJc\n-----END PGP MESSAGE-----\n"
```


As the ciphertext has additional text that is not needed, the command below removes the unnecessary parts.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ cat 3_1_1-DeltaZhang00-to-ALICEKerr000-msg.out | jq -r .SESSInit.
c_i > Ciphertext.txt
```

Output:

```
-----BEGIN PGP MESSAGE-----

hQEMA9aIh3UhmnlwAQf+MJYyBZCECKJE7rjVo/XD17wFYMj7CRD16t2dvJxV6t9N
U/SsIIoWU5eJsnUdWiHCr76R914Vk4HZY3smOT99gQIIfE0UOc6W2jCog1dxA/Y1
2aiNVkCYwnlBNgjie0fPLeEiMrr9zM6AKycuXYve4R41nkfNFt3KEoRZvdDFuJwy
2X5v4RNlX29y/PTa8bw/XNhnfzhnnPRu/ZuqeamtqIRz4r1l+1lEbTQhmoNB2pv1
KGIi2EO6wjJXgq0JMbhMYwCv/C+62ndwSU1FrlNRnL295ojvd7XH0DtlKTtpdc9M
+QNusY5q0XmDOK0AADdy7zwCJlPhoLBUD4MeptU1+tJeATELmfiTAEU1ctZ9UYMz
yOXZdMJ3RkllXquK0iYe76xB3CLcnzHfSqT31BpBm/nDQd+II0J3qSscJ+DxNeeZ
P34QoaUkDXdAc6XjcIvQsw+LkUsIoV7ZljU19qK4Vg==
=GtJc
-----END PGP MESSAGE-----
```

Next, using the command below, “ALICEKerr000”’s secret key is imported.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ gpg --import ALICEKerr000_e.key
gpg: key D6888775219A7970: public key "ALICEKerr000_e (ALICEKerr000 Encryption Key)" imported
gpg: key D6888775219A7970: secret key imported
gpg: Total number processed: 1
gpg:               imported: 1
gpg:       secret keys read: 1
gpg:   secret keys imported: 1
```

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ gpg --import ALICEKerr000_s.key
gpg: key EB1AE2D953D4CAE3: public key "ALICEKerr000_s (ALICEKerr000 Signature Key)" imported
gpg: key EB1AE2D953D4CAE3: secret key imported
gpg: Total number processed: 1
gpg:               imported: 1
gpg:       secret keys read: 1
gpg:   secret keys imported: 1
```

With the secret key, the decryption can proceed smoothly as it would have to verify the signature’s authenticity first.

Using the command below and “ALICEKerr000”’s obtained secret key, the ciphertext sent by “DeltaZhang00” is decrypted.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ gpg --output plain-dec.txt --decrypt Ciphertext.txt
gpg: encrypted with 2048-bit RSA key, ID D6888775219A7970, created 2024-04-06
      "ALICEKerr000_e (ALICEKerr000 Encryption Key)"
```

Output:

```
1 Ù©Ó€A
2 ÇFr³H   ýaÊ□□ÞQ□□öÁ□□o¶ü□□Ì□□□□Ä□□
```

Finally, the plaintext is converted to a readable format (Hexadecimal) using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.1$ xxd -ps plain-dec.txt > plaintext.txt
```

Output:

```
1 d9a9d3a4410dc74672b34809fd61ca85de511af6c19e6fb6fc8fcc1e0c1b
2 c482
```

With that, the gsk (in Hexadecimal format) has been obtained.

I hope you find this email helpful in understanding how to complete the group session initialization.

Regards,

Braden

SubTask 3.2: Confidentiality of the Group Session Initialization Protocol

To: reyes2093@gmail.com

Subject: Confidentiality of the Group Session Initialization Protocol

Dear Reyes,

I am writing this email to explain about the potential threats and information available to passive and active attackers aiming to attack the user enrolment and group session initialization protocols.

User Enrolment: Passive Attack

I believe the system is safe from passive attacks and successfully maintains confidentiality. This is because not only is the user's ID private and only known to the user who set it, the secret key is stored in the system and hidden from outsiders. This makes it difficult for outsiders to eavesdrop on conversations to pick up any sensitive information.

User Enrolment: Active Attack

The system is relatively safe from active attacks as well as the RSA signature and encryption key pairs are 2048 bits long. The length of the key pairs protects the system from brute force attacks, maintaining confidentiality.

Group Session Initialization: Passive Attack

Just like the user enrolment protocol, the group session initialization protocol is quite safe from passive attacks as well as the messages are encrypted.

Group Session Initialization: Active Attack

The system is at risk of an active attack due to how the user's keys are handled. During the user's enrolment, an attacker could alter the cert list of the user currently enrolling by replacing the user's public encryption with their own. The group session initialization protocol would be unable to validate the user's identity as their digital signature has not been established yet.

Once the attacker has replaced the target's public key, the cert list is sent to the server and signed. This would allow the attacker to impersonate themselves as the target and compromise the gsk by decrypting it with their own private key when they receive a message sent by the group initiator.

I will demonstrate this method of attack below.

Attack Demonstration

First, using the commands below, "DeltaZhang00"'s (the initiator's) key pairs are imported.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ gpg --import DeltaZhang00_e.key
gpg: key 5D12B09DB1ABA04B: public key "DeltaZhang00_e (DeltaZhang00 Encryption Key)" imported
gpg: key 5D12B09DB1ABA04B: secret key imported
gpg: key 663BAE9734C9B6BF: public key "DeltaZhang00_e (DeltaZhang00 Encryption Key)" imported
gpg: key 663BAE9734C9B6BF: secret key imported
gpg: Total number processed: 2
gpg:               imported: 2
gpg:       secret keys read: 2
gpg:   secret keys imported: 2
```

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ gpg --import DeltaZhang00_s.key
gpg: key 6FBF3BA61C01E769: public key "DeltaZhang00_s (DeltaZhang00 Signature Key)" imported
gpg: key 6FBF3BA61C01E769: secret key imported
gpg: key ED8CFA42BBFA192A: public key "DeltaZhang00_s (DeltaZhang00 Signature Key)" imported
gpg: key ED8CFA42BBFA192A: secret key imported
gpg: Total number processed: 2
gpg:               imported: 2
gpg:       secret keys read: 2
gpg:   secret keys imported: 2
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$
```

Next, using GPG, a key-pair is generated.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ gpg --full-generate-key
```

```
Please select what kind of key you want:
   (1) RSA and RSA (default)
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 2048
Requested keysize is 2048 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1
Key expires at Sun 12 May 2024 22:03:04 AEST
Is this correct? (y/N) y
```

```
You need a user ID to identify your key; the software constructs the user ID
from the Real Name, Comment and Email Address in this form:
    "Heinrich Heine (Der Dichter) <heinrichh@duesseldorf.de>"

Real name: demo
Name must be at least 5 characters long
Real name: demonstration
E-mail address: demo@gmail.com
Comment: demo email
You selected this USER-ID:
    "demonstration (demo email) <demo@gmail.com>"
```

Type of Key: RSA and RSA

Key Length: 2048 bits

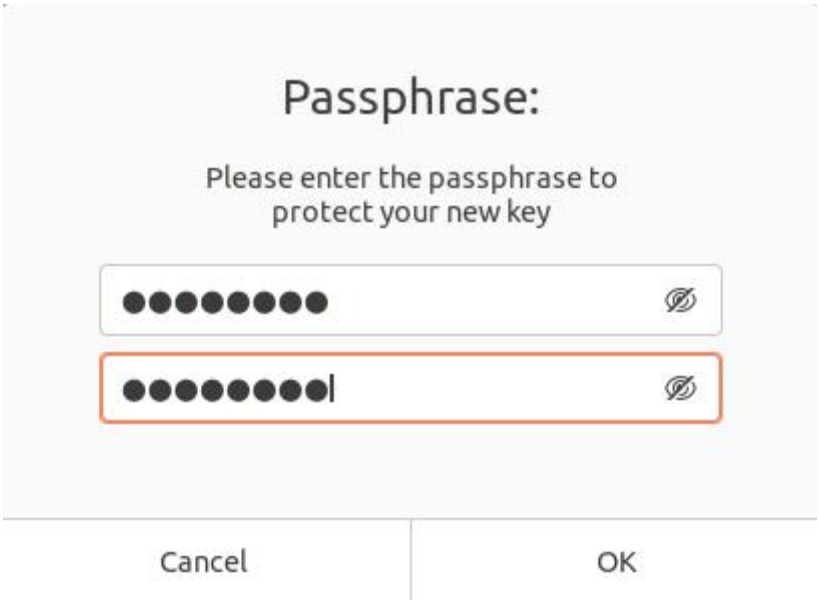
Expiration date: 1 day

Real Name: demonstration

Email Address: demo@gmail.com

Comment: demo email

After the key is created, the user would enter a password only they know to protect their new key. This ensures that only the user has access to their secret key.




```
public and secret key created and signed.

pub   rsa2048 2024-05-11 [SC] [expires: 2024-05-12]
      2AB8888A9EE41E521C489EA110E8425BEFCCAD94
uid                      demonstration (demo email) <demo@gmail.com>
sub   rsa2048 2024-05-11 [E] [expires: 2024-05-12]
```

The public and secret key are now created.

Next, the public key is stored in a variable using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ newcert=`gpg --export --armor "demonstration" | awk '{printf "%s\
\n", $0}'`
```

After storing the variable, the public encryption key of “PrincePhi123” is replaced with the public key generated prior using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ jq ".[1].cert_e.pk |= (sub(\".*\";\"${newcert}\"; \"m\"))" CertRe
sp-PrincePhi123-CharlieLake0-ALICEKerr000.json > attack.json
```

“PrincePhi123”’s public key has been successfully replaced.

After that, the command below is used to simulate “DeltaZhang00” initiating a session with “PrincePhi123”, “CharlieLake0” and “ALICEKerr000”.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ ./PeerHelper_x86 --peer PrincePhi123 CharlieLake0 ALICEKerr000 --
certresp attack.json
```



Next, the cihpertext sent to “PrincePhi123” (the target) is extracted using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ jq .SESSInit.c_i 3_2-DeltaZhang00-to-PrincePhi123-msg.out > Ciphe
rtext.txt
```

Output:

```
"-----BEGIN PGP MESSAGE-----\n\nhQEMA34hy+O1dF1ZAQgAlNEJE6hqhNH4Q+7gtxtgPib/G1kc7VOxKgklWHYJ55ah\nZTb57HAOqRvDa/eTlHx7TLwCQDodGyT97v
TNB7bE+YHqrsgx71WNf4J2Uge4oC3i\nxNUJcbbTRXzHsMA0SyPbttikO62fbhmugODfQY0mlenQnQyRMLyIQPib6ZhSYOnd\nk+2oNaUaV+ixl31fzaf6gr+/C9M228c3hf
bcmJ13fmrllF0pMynWb2OjM4LVIIPi\nC2yMWAAvMuICnjBqa4vwssS158+VqNn2qNtLIDUE/4PDkbCsBv/+dmmIsE56V2f4\nHcJTbeobYA5NOk177b0n6aAWkso0rV+LHd
mQAzt8ydJeAZRttjIziu0LCyN4A5np\nEvGetWwO/HDsaeKmMPUiiFJECZ2DhF+1i+kMDwB0cprDwpgylWgeao79tYmMeHGN\n/N5trsO7p86HPkEwZRAKURiWCTr+K+2rsU
at7ZC9Ug==\n=qa8W\n-----END PGP MESSAGE-----\n"
```

The ciphertext is then converted to a suitable format (asc) using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ cat Ciphertext.txt | jq -r . Ciphertext.txt > Ciphertext.asc
```

Output:

```
 1 -----BEGIN PGP MESSAGE-----
 2
 3 hQEMA34hy+O1dF1ZAQgAlNEJE6hqhNH4Q+7gtxtgPib/G1kc7VOxKgklWHYJ55ah
 4 ZTb57HAOqRvDa/eTlHx7TLwCQDodGyT97vTNB7bE+YHqrsgx71WNf4J2Uge4oC3i
 5 xNUJcbbTRXzHsMA0SyPbttikO62fbhmugODfQY0mlenQnQyRMLyIQPib6ZhSYOnd
 6 k+2oNaUaV+ixl31fzaf6gr+/C9M228c3hfbcmJ13fmrllF0pMynWb2OjM4LVIIPi
 7 C2yMWAAvMuICnjBqa4vwssS158+VqNn2qNtLIDUE/4PDkbCsBv/+dmmIsE56V2f4
 8 HcJTbeobYA5NOk177b0n6aAWkso0rV+LHdmQAzt8ydJeAZRttjIziu0LCyN4A5np
 9 EvGetWwO/HDsaeKmMPUiiFJECZ2DhF+1i+kMDwB0cprDwpgylWgeao79tYmMeHGN
10 /N5trsO7p86HPkEwZRAKURiWCTr+K+2rsUat7ZC9Ug==
11 =qa8W
12 -----END PGP MESSAGE-----
```

Using the command below, the ciphertext is decrypted to obtain the gsk.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ gpg --output gsk.txt --decrypt Ciphertext.asc
gpg: encrypted with 2048-bit RSA key, ID 7E21CBE3B5745D59, created 2024-05-11
      "demonstration (demo email) <demo@gmail.com>"
```

Output:

```
1 [00]95Ò[00]8A.mäQ2a²ÜÄÓ[00]8Dµ[00]94Ò°[00]81[00]97[00]1C[00]92[00]8CÔç íEž":
```

Finally, the gsk is converted to Hexadecimal format using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/3.2$ xxd -ps gsk.txt > Hexgsk.txt
```

Output:

```
1 95d28a2e6de55132ad61b2dcc4d38db594d2ba81971c928cd4a2a0ed45b8
2 223a
```

With this, the gsk has been compromised by the attacker.

I hope you find this email helpful in understanding about the potential threats to the group session initialization protocol.

Regards,

Braden